PRD — InsightFlow v2

Status: Build spec **Constraint:** Free software and free tiers only **Supersedes:** InsightFlow Project Summary & Roadmap v1

---

0. What changed from your v1 doc, and why

Your v1 had the right instinct in three places and the wrong instinct in three others. This spec keeps the former and cuts the latter.

Kept from v1:

- **Config-driven, dataset-agnostic design.** "Works with any business dataset" is the strongest idea in your document. Most projects hardcode one schema. A config file that declares metrics, date columns, and thresholds means you can demo the same system on sales, inventory, and support-ticket data in one interview. That's a real differentiator — lead with it.
- **KPI layer.** v1 had this and my earlier spec didn't. It matters: raw column anomalies ("units_sold dropped") are less useful than derived-metric anomalies ("gross margin % dropped while revenue held" — which means cost moved). Derived metrics are where the actual business signal lives.
- **SQL persistence + historical reports.** MySQL is more work than SQLite but SQL is a hiring reality and "I designed the schema" is a real interview topic. Keeping it, with a caveat in §7.

Cut from v1:

- **Power BI *and* Tableau *and* Streamlit.** Three dashboarding tools for one project is resume padding, and reviewers read it that way. Worse: Power BI free tier can't publish to the web, so it becomes a screenshot in a README — undemonstrable. **Pick Streamlit only.** It's free, deployable on Streamlit Community Cloud, and gives you a live URL to put on the resume. One working dashboard beats three claimed ones.
- **Role-based login.** You have no users. Auth with no threat model is security theater, and a hand-rolled login is more likely to become an interview liability ("how do you store passwords?") than an asset. Cut it. If you want the credential story, do secrets management properly instead (§6) — that's the part that actually gets asked about.

- **Scikit-learn as listed.** I suspect this is here to reach for `IsolationForest`. Don't. On univariate time series it underperforms a robust z-score, needs tuning you can't justify, and you'll be asked why you chose it. Use it in exactly one place where it earns its keep (§4, FR-3 multivariate check) or not at all.
- **"Root cause analysis."** As written this is undeliverable — you cannot infer causation from one spreadsheet. Renamed to **driver attribution** and scoped honestly in FR-4: correlation and decomposition, labeled as hypotheses, never as causes. Overclaiming this in an interview is the fastest way to lose the room.
- **The timeline.** 3–4 weeks assumes zero rework, which never happens. Sequence matters more than dates — see §9.

Added to both:

- Human approval gate before any email leaves the system.
- Numeric-grounding check on all LLM output.
- A labeled evaluation set with published precision/recall. **This is the single highest-value item in this document.**

---

1. Product summary

InsightFlow ingests a business metrics file, cleans and validates it, computes configured KPIs, detects statistically significant changes, attributes probable drivers, generates a plain-language summary grounded in the actual numbers, routes it through human approval, and emails approved findings to stakeholders. All runs, flags, approvals, and sends are logged and browsable in a dashboard.

One-sentence positioning: a configurable anomaly-alerting pipeline for tabular business data, with measured detection accuracy and a human gate before anything reaches a stakeholder.

Explicitly not: an LLM that reads a spreadsheet and writes a report. Detection is statistical; the LLM only narrates pre-verified findings.

---

2. Design principles

These are the answers to the three hardest questions an interviewer will ask. Build so they stay true.

1. **Detection is statistical, narration is LLM.** LLMs cannot reliably spot outliers in tables — they hallucinate both the anomaly and the explanation. Separating the stages makes each independently testable.
2. **Nothing sends without a human.** An automated system that emails wrong numbers to managers is worse than no system. The approval gate is a design feature; say so out loud.
3. **Every number in the output traces to a source row.** Enforced programmatically, not by prompt instruction (FR-5).
4. **Config over code.** New dataset = new YAML file, no code changes. This is what makes it a platform rather than a script.

---

3. Users

Role	Need
Analyst / approver	Review flagged anomalies, verify against source, approve or reject with a reason
Manager / recipient	Short email: what moved, by how much, probable drivers, link to detail
Operator (you)	See run history, detection accuracy over time, and failures

---

4. Functional requirements

FR-1: Config-driven dataset definition

A single YAML per dataset declares everything:

```
dataset: retail_sales
source: data/sales.xlsx
sheet: Sheet1
date_column: order_date
grain: daily
raw_metrics:
  - name: revenue
    column: total_amount
    agg: sum
```

```
  - name: units
    column: quantity
    agg: sum
  - name: cogs
    column: cost
    agg: sum
kpis:
  - name: gross_margin_pct
    formula: (revenue - cogs) / revenue * 100
  - name: avg_order_value
    formula: revenue / order_count
dimensions: [region, category, channel]
detection:
  gross_margin_pct:
    z_threshold: 2.5
    min_pct_change: 3.0
  revenue:
    z_threshold: 3.0
    min_pct_change: 10.0
business_context:
  revenue: "Total gross sales before returns. Owner: Sales."
  gross_margin_pct: "Margin after COGS. Below 30% triggers review."
known_events:
  - { date: 2025-11-28, label: "Black Friday - expect 4x volume" }
recipients: [manager@example.com]
```

Acceptance test: ship with three config files for three genuinely different datasets (retail sales, inventory, support tickets). Demo all three. If adding the third required a code change, FR-1 has failed.

FR-2: Ingest, clean, validate

- Accepts `.xlsx`, `.csv`. Rejects `.xls` with a message telling the user to re-save.
- **Must survive:** merged cells, multi-row headers, blank rows/columns, thousands separators, currency symbols, percentage strings, `N/A` / `NULL` / `-` /empty as missing, mixed date formats, trailing whitespace, duplicate rows, negative values where impossible.
- Validation gate — abort the run and log the reason if any fail:
  - declared date column parses as dates for ≥95% of rows
  - all declared metric columns exist and are numeric after cleaning
  - ≥30 periods of history at the configured grain

    - no duplicate date+dimension keys after aggregation
- **On any validation failure: abort, log, send nothing.** Never proceed on partial data.
- Log a cleaning report per run: rows in, rows dropped and why, coercions applied. This report is a deliverable — it answers "how do I know your cleaning didn't silently corrupt the data?"

FR-3: KPI computation

- Evaluate KPI formulas from config against aggregated raw metrics (use a restricted expression evaluator — `asteval` or similar; do **not** use bare `eval()`, and say why in your README).
- Handle divide-by-zero and nulls explicitly: KPI is null for that period, not zero, not infinity. Silent zeros are how these systems produce confidently wrong alerts.
- Compute at the configured grain and, where dimensions are declared, per dimension value.

FR-4: Anomaly detection

Per metric and KPI, in order:

1. **Baseline:** trailing window (default 30 periods, per-metric override).
2. **Seasonality:** if a weekly/monthly pattern is detectable, decompose with `statsmodels.STL` and run detection on the residual. Otherwise use the raw series. Without this you flag every weekend dip.
3. **Known-event suppression:** dates in `known_events` are excluded from flagging and from baseline calculation. This is the cheapest large precision win available and almost nobody does it.
4. **Rules** — flag if any fire:
    - **Point outlier:** robust z-score (median/MAD) > threshold
    - **Period-over-period:** % change exceeds per-metric threshold
    - **Level shift:** rolling mean of last N periods vs prior N exceeds threshold — catches sustained changes that never look like a spike
    - **Composite (optional, this is where scikit-learn earns its place):** `IsolationForest` over the joint KPI vector to catch combinations that are individually normal but jointly abnormal — e.g. volume up, margin down, AOV flat. Build this **last**, and only after univariate detection is measured. If it doesn't beat

univariate on your eval set, cut it and say in your README that you tested it and it didn't earn its complexity. That sentence is worth more than the feature.

5. **Output per flag (structured):** `metric, date, dimension, actual, expected, abs_delta, pct_delta, z_score, rule, direction, severity`.

Zero LLM involvement in this stage.

FR-5: Driver attribution (renamed from "root cause")

When a metric is flagged, mechanically decompose where the change came from — do not ask the LLM to guess:

- **Dimensional contribution:** for each declared dimension, compute each value's contribution to the total delta. "78% of the revenue drop came from the West region."
- **Formula decomposition:** for KPIs, identify which input moved. "Margin fell because COGS rose 14% while revenue was flat."
- **Correlated metrics:** other metrics that moved in the same window, with correlation coefficient. **Label these as correlations, not causes, in the output schema and in the email.**

This is arithmetic and it's defensible. It's also more genuinely impressive than LLM speculation, because it's right.

FR-6: Narration

- LLM input: the FR-4 flag record + FR-5 attribution + `business_context` from config. Nothing else. Never the raw file.
- Enforced JSON output: `{ headline, explanation, drivers[], confidence: high|medium|low }`.
- **Grounding check (hard requirement):** extract every numeral from the LLM output and assert each appears in the input record. On failure, discard the narration and fall back to a deterministic template. Log the rejection. Report the rejection rate in your evals — it's a metric almost no one measures and it will be noticed.
- If the LLM API is down or over quota, fall back to templates and **still deliver the alert**. Detection must never depend on the LLM being available.

FR-7: Deduplication

- Suppress a flag if the same `(metric, dimension, rule, direction)` alerted within the cooldown window (default 7 days).

- Suppressed flags are logged, not sent.
- Re-running on an identical file (same SHA-256) produces no new emails. Idempotency is testable — write the test.

FR-8: Approval gate

- Post-detection, run state = `pending_approval`.
- Approver reviews in the Streamlit dashboard: per-anomaly approve/reject, with source rows and the chart visible alongside.
- Rejection requires a reason: `false_positive | known_cause | data_error | not_material`.
- **Rejection reasons are your ongoing labeled dataset.** Over time, plot precision by metric from real approver decisions. That chart is the best artifact this project can produce.
- No approval in 48h → run expires, logged as expired, nothing sent.

FR-9: Email

- Approved anomalies only, ordered by severity.
- Per anomaly: headline, explanation, driver attribution, and a table of `metric | date | actual | baseline | delta` so every claim is checkable.
- Correlation-based drivers explicitly labeled as hypotheses.
- Link to the dashboard detail view.
- **Zero approved anomalies → send nothing.** No "all clear" emails; they train recipients to ignore you.

FR-10: Dashboard (Streamlit only)

Four pages, no more:

1. **Review queue** — pending anomalies, approve/reject
2. **Metric explorer** — time series per metric with flags and baseline band overlaid
3. **Run history** — audit log, cleaning reports, failures
4. **Detection health** — precision/recall from eval set + rolling precision from real approver decisions

Page 4 is the one that gets you hired. Most dashboards show data; this one shows that you measure your own system.

FR-11: Persistence & audit

Schema (MySQL):

- `runs` — id, dataset, file_hash, started_at, status, rows_in, rows_dropped, error
- `metrics_timeseries` — run_id, dataset, date, dimension, metric, value
- `anomalies` — id, run_id, metric, date, dimension, actual, expected, pct_delta, z_score, rule, severity, status, narration_json, grounding_check_passed
- `approvals` — anomaly_id, decision, reason, decided_at
- `sends` — run_id, recipients, sent_at, anomaly_ids

Index on `(dataset, metric, date)` and `(run_id)`. Be ready to explain both.

FR-12: Export

PDF and Excel export of a run's findings. Use `reportlab` or `weasyprint` for PDF, `openpyxl` for Excel. Low priority — build last, it's table stakes, not a differentiator.

---

5. Out of scope (v2)

State these in the README so it reads as scoping, not as gaps:

- Live database/API connectors (file ingest only)
- Multi-tenancy, RBAC, user accounts
- Forecasting
- Natural-language querying / "chat with your data"
- Streaming or sub-daily grain
- Mobile push notifications

---

6. Non-functional requirements

- **Idempotency:** same file hash → no duplicate sends. Tested.

- **Failure loudness:** every failure logged with cause; no silent partial success. LLM failure degrades to templates; ingest failure aborts entirely.
- **Secrets:** DB credentials, LLM key, SMTP password from environment only. `.env` gitignored from commit #1. A committed API key in history is an instant credibility loss if anyone reads your repo.
- **Data egress:** document exactly what leaves the machine — flagged rows and metric names to the LLM API, never the full file. Put this in the README under a "Data handling" heading.
- **Cost:** stays inside free tiers. Batch narration calls; cache by flag hash so re-runs don't re-spend quota.
- **Performance:** 100k rows end-to-end under 2 minutes. Not a real constraint at this scale — but if you exceed it, you're doing something pathological (row-wise `.apply()`, most likely).

---

7. Stack

Layer	Choice	Justification
Language	Python 3.11+	Only ecosystem with both mature stats and LLM tooling
Ingest	`pandas`, `openpyxl`	Standard; handles the messy-Excel cases
Expressions	`asteval`	Safe KPI formula evaluation without `eval()`
Detection	`numpy`, `scipy`, `statsmodels` (STL)	Free, defensible, explainable in an interview
Composite detection	`scikit-learn` `IsolationForest`	**Optional, built last, kept only if it beats univariate on evals**
LLM	Google Gemini or Groq free tier	Genuinely usable free quota + structured output support
Database	MySQL 8 (local) or Neon/Supabase free tier	SQL is a hiring reality; managed free tier means a live demo URL works
Dashboard	Streamlit + Streamlit Community Cloud	Free, deployable, gives you a live link

Email	Gmail SMTP + app password	Free, no signup friction
Scheduling	GitHub Actions cron	Free, and the workflow file is your deployment story
Tests	`pytest`	Non-negotiable — see §8
Lint/format	`ruff`	One tool, zero config, signals hygiene

Deliberately excluded, with reasons ready: Power BI and Tableau (redundant, undemoable on free tier), LangChain (linear stateless pipeline — adds abstraction, removes no work), vector DB (nothing to retrieve), Airflow (one DAG with seven tasks doesn't justify a scheduler service), Docker Compose multi-service (nothing to orchestrate).

If asked why you excluded something, "I evaluated it and it didn't earn its operational complexity" is a senior answer. "I didn't know about it" is not. Know your exclusions.

---

8. Evaluation — the part that makes this resume-worthy

Build this before the LLM layer. Without it the rest is a demo.

Labeled set: 12–24 months of metric data (real or synthetic). Inject:

- 10 point spikes of varying magnitude
- 8 sustained level shifts
- 6 gradual drifts
- 6 dimension-localized anomalies (one region only)
- **8 seasonal patterns that are NOT anomalies** — false-positive traps
- 4 known-event spikes that should be suppressed by FR-4 step 3

Measure and publish:

Metric	Target
Recall (injected anomalies caught)	≥ 85%
Precision (flags that are real)	≥ 80%
False positives on seasonal traps	0

Known-event suppression rate	100%
Numeric-hallucination rejection rate	< 5%
Dimension attribution accuracy	≥ 90%

Tune thresholds against this set, record the precision/recall curve, and pick an operating point deliberately. **For alerting, favor precision** — noisy alerts get filtered into a folder and the system dies. Be able to say that.

Commit `evals/results.md` with the numbers, the curve, and a short "where it fails" paragraph. Honest numbers beat inflated ones; "here's the failure mode I couldn't solve" is a stronger interview answer than "it works great."

---

9. Build order

Sequence is load-bearing. Do not reorder.

#	Deliverable	Done when
1	Repo, `ruff`, `pytest`, `.env` pattern, MySQL schema	Migrations run clean, no secrets in git
2	Config loader + 1 dataset config	Config parses, validation errors are clear
3	Ingest + cleaning + validation gate	Passes tests against 5 deliberately-broken files you author
4	KPI engine	Divide-by-zero and null cases unit-tested
5	Detection: 3 univariate rules	Unit test per rule
6	**Eval harness + labeled dataset**	Baseline precision/recall recorded
7	Seasonality + known-event suppression	Re-run evals, show the false-positive drop
8	Driver attribution	Contributions sum to the total delta (assert this)
9	LLM narration + grounding check	Rejection rate measured
10	Dedup + idempotency	Same-file re-run sends nothing (tested)
11	Streamlit: review queue + explorer	Approve/reject writes to DB

12	Email send	Only approved items; correct on empty set
13	Detection-health page	Rolling precision from real approvals renders
14	GitHub Actions cron	Green scheduled run
15	Configs 2 and 3 (different datasets)	**Zero code changes required**
16	Composite `IsolationForest`	Kept only if it beats univariate; documented either way
17	PDF/Excel export	—
18	README: architecture, evals, data handling, limitations	—

Step 6 before step 9 is the whole point. Most people build the LLM first because it's fun, then find their detection is unmeasured and the demo only works on one hand-picked file. Step 15 is what converts "a script" into "a platform" — do not skip it.

---

10. Known limitations — write these yourself in the README

- Detects *that* a metric changed and *where* the change concentrated. Does not establish causation; correlated drivers are hypotheses, labeled as such.
- File ingest only; no live connectors.
- Requires ≥30 periods of history per metric.
- Thresholds tuned on one dataset; require retuning per deployment.
- Single-tenant, no auth — deliberate scoping, not oversight.
- LLM narration is a convenience layer; the system degrades to templates and remains fully functional without it.

This section is a stronger hiring signal than any feature above. It shows you know where your own system breaks.

---

11. Resume framing

> **InsightFlow** — Configurable anomaly-detection and alerting pipeline for tabular business data. Statistical detection (robust z-score, STL residual analysis, level-shift detection, known-event suppression) with mechanical driver attribution and LLM narration gated by a numeric-grounding check and human approval before send. Achieved __% recall / __% precision on a 42-anomaly labeled eval set with zero false positives on seasonal patterns. Config-driven — deployed across three unrelated datasets with no code changes. Python, pandas, statsmodels, MySQL, Streamlit, GitHub Actions.

Fill in real numbers. A bullet with no measurement in it is a bullet worth cutting.